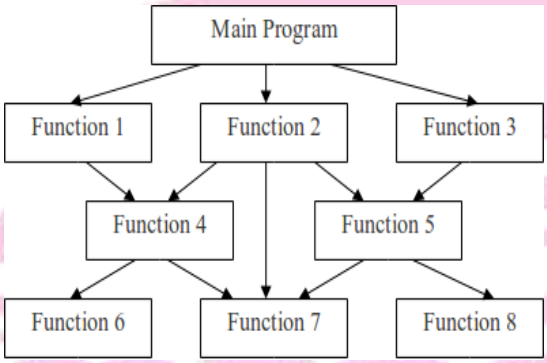
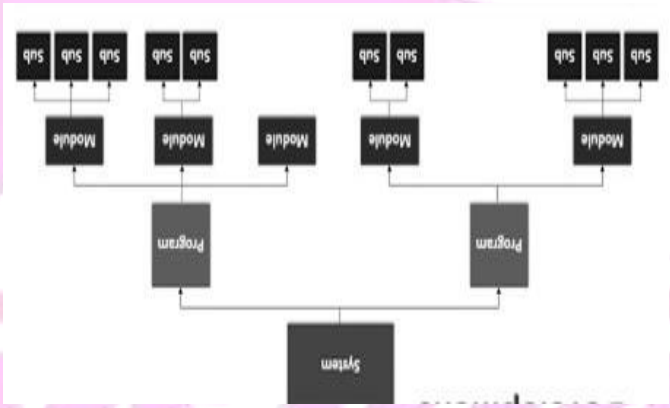
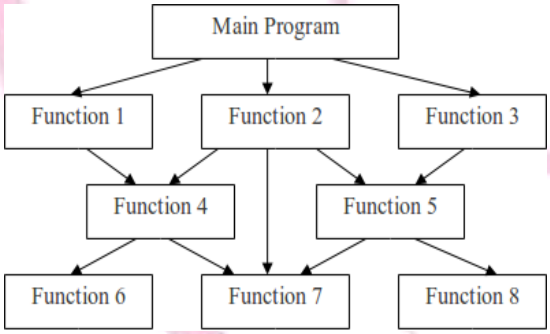
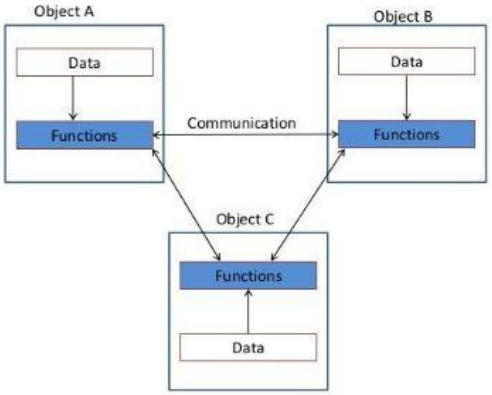
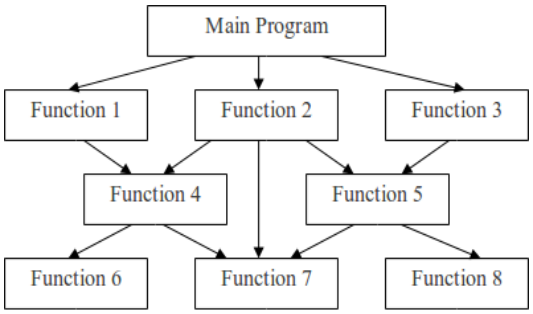
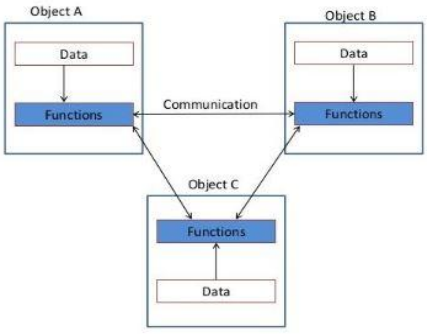
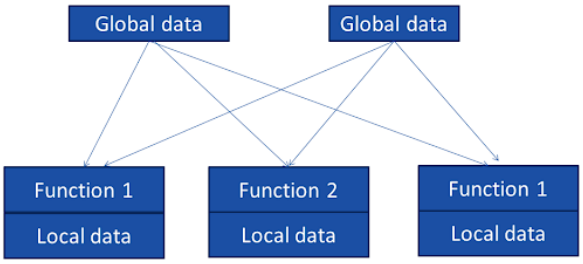
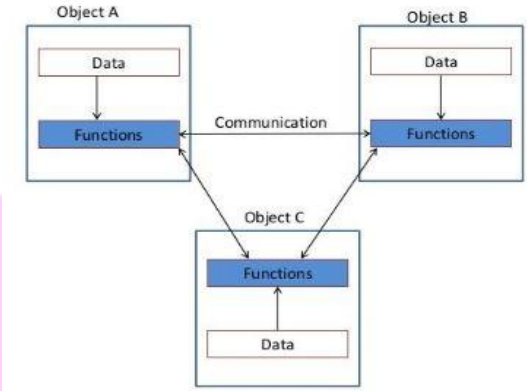
Syllabus of OOPs

Review of Fundamentals of Procedural Programming, Class and Objects, Data Abstraction, Information Hiding & Encapsulation, Constructors, destructors, and object creation, Name space and references , Class Methods , Methods Overloading , Inheritance , Polymorphism , Abstract Classes, Abstract Methods , Exceptions , Exception Handling.●

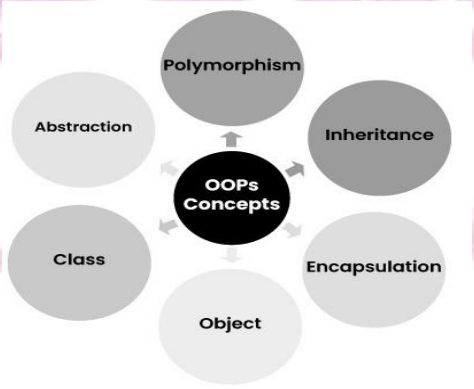
Procedural oriented programming Vs Object Oriented Programming

SNo.	POPS	OOPS
1.	Top down approach 	Bottom up approach 
2.	Based on functions, procedures or subroutines 	Based on real world objects which contains data and method. 
3.	Complex to implement large applications. 	Easy to implement large projects. 

4.	**It is less secure** 	**Highly secure** 
5.	**Overloading is allowed.**	**Overloading is not allowed**
6.	**Less modularity; functions are interdependent.**	**High modularity; objects can be reused and modified independently.**
7.	**C, Pascal, Fortran.**	**Java, C++, Python, C#.**

Object oriented programming concepts

- ✓ Class
- ✓ Objects
- ✓ Inheritance
- ✓ Encapsulation
- ✓ Abstraction
- ✓ Polymorphism



Class

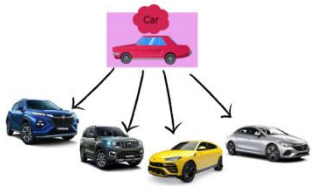
- A class is a user-defined blueprint or prototype used to create objects.
- It is a logical entity containing data members (variables defining the object's state) and member functions (methods defining the object's behavior).
- Memory is not allocated to classes, as they have no physical existence, unlike objects which are physical entities.
- Classes cannot be manipulated directly due to the lack of memory allocation, but objects can be.
- Access specifiers in classes control the visibility of data members and methods. By default, members are private if no access specifier is mentioned.
- Member functions can be defined in two ways:
  - **Inside the class**: Defined directly within the class definition.
  - **Outside the class**: Defined using the scope resolution operator (::) (in C++)with the class name and function name.

In Python, member functions cannot be defined outside the class. However, you can use functions outside the class and pass the class object as a parameter to simulate similar behavior.

Java	CPP	Python

class ClassName
{
// Data members (variables)
int dataMember;
 	
// Constructor
ClassName()
{
// Initialization code
}
 	
// Member function
void memberFunction1()
{
// Function code
}
 	
// member function declaration
void memberFunction2();
}
 	
// Function definition outside the class
void ClassName:: memberFunction2 ()
{
// Function code
}
 	
public static void main(String[] args)
{
// Creating an object of the class
ClassName obj = new ClassName();
obj.memberFunction1();
obj.memberFunction2();
}
}
	class ClassName
	{
	private:
	// Data members
	int dataMember;
	
	public:
	// Constructor
	ClassName()
	{
	// Initialization code
	}
	
	// Member function
	void memberFunction1()
	{
	// Function code
	}
	
	// Member function declaration
	void memberFunction2 ();
	};
	
	// Function definition outside the class
	void ClassName:: memberFunction2 () {
	// Function code
	}
	
	int main()
	{
	// Creating an object of the class
	ClassName obj;
	obj.memberFunction1();
	obj.memberFunction2();
	return 0;
	}
		class ClassName:
		
		# Constructor
		def __init__(self):
		
		# Initialization code
		self.data_member = 0
		
		# Member function
		def member_function1(self):
		# Function code
		pass
		
		# Function outside the class
		def member_function2(ClassName):
		# Function code
		pass
		
		# Creating an object of the class
		obj = ClassName()
		obj.member_function1()
		member_function2(obj)

Objects



- An object is a real-world entity, like a chair, car, or mobile.
- Objects are created to access the data and methods of a class.
- They allow access to the class's data and member functions.
- Data defines an object's state, and methods define its behavior.
- Objects encapsulate data and behavior, ensuring better modularity and reusability in programs.
- Memory is allocated to objects, and their size depends on the class's data members.
- Multiple objects and array-type objects can be created for a class.
- Each object has its own copy of the class's non-static data members, ensuring they maintain independent states.
- Objects can also interact with one another by calling each other's methods or accessing shared data.

Subscribe Infeepedia youtube channel for computer science competitive exams
Download Infeepedia app and call or wapp on 8004391758

Java	CPP	Python

class Car
{
String model;
int year;
Car(String model, int year)
{
this.model = model;
this.year = year;
}
void display()
{
System.out.println("Model: " + model + ", Year: " + year);
}
}
public class Main
{
public static void main(String[] args)
{
Car car1 = new Car("Toyota", 2020);
Car car2 = new Car("Honda", 2022);
car1.display();
car2.display();
}
}
	#include <iostream>
	using namespace std;
	class Car
	{
	public:
	string model;
	int year;
	Car(string m, int y)
	{
	model = m;
	year = y;
	}
	void display()
	{
	cout << "Model: " << model << ", Year: " << year << endl; }
	};
	int main()
	{
	Car car1("Toyota", 2020);
	Car car2("Honda", 2022);
	car1.display();
	car2.display();
	return 0;
	}
		class Car:
		def __init__(self, model, year):
		self.model = model
		self.year = year
		def display(self):
		print(f"Model: {self.model}, Year: {self.year}")
		
		car1 = Car("Toyota", 2020)
		car2 = Car("Honda", 2022)
		
		car1.display()
		car2.display()

Constructor

- **Constructors** are special member functions used to initialize objects of a class.
- They have the same name as the class and no return type(in C++ and java).
- Automatically invoked when an object is created, constructors set initial values for the object's attributes.
- They can be **overloaded** but cannot be declared **virtual** or inherited.
- A constructor's address cannot be referenced.
- Constructors must be placed in the **public** section of a class.
- They can be defined either inside or outside the class declaration.
- In **Java**, constructors can call other constructors in the same class using this().
- In **Python**, the constructor is defined using the **__init__()** method.
- In Java and Python, constructors are tightly integrated with the class structure, ensuring encapsulation and simplicity in object creation and cannot declare or define outside the class.
- In C++ constructors can be declared or defined outside the class.

Syntax of Constructors:	**Syntax of Declaring a Constructor**	**Syntax of Declaring a Constructor**

<class-name> (list-of-parameters);
 	
Syntax for Defining the Constructor Within the Class
<class-name> (list-of-parameters)
{
// constructor definition
}
	<class-name>(list-of-parameters) {
	// Constructor definition
	}
		def __init__(self, list_of_parameters):
		# Constructor definition

Subscribe Infeepedia youtube channel for computer science competitive exams
Download Infeepedia app and call or wapp on 8004391758

Syntax for Defining the Constructor Outside the Class

```
<class-name>::<class-name>(list-of-parameters)
{
   // constructor definition
}
```

Types of Constructor

Based on the situations they are being used constructor is classified in 3 types.

1. **Default Constructor**
2. **Parameterized Constructor**
3. **Copy Constructor**

1. Default constructor

- A **default constructor** is a constructor that takes no arguments and has no parameters. ‘
- It initializes the object's attributes with default values.
- It is also called a **zero-argument constructor**.
- Automatically provided by the compiler if no constructor is explicitly defined.
- Used to initialize objects with default values.
- If a class has at least one user-defined constructor, the compiler does not generate a default constructor.
- **In Java and C++,** the compiler automatically provides a **no-argument constructor** if no constructors are explicitly defined. If a user-defined constructor is present, the default constructor is not generated.
- **In Python,** the default constructor is defined using the **__init__()** method with no parameters except self. If no constructor is explicitly defined, Python does not generate a default constructor automatically.

Java

```
class Student
 {
String name;
 int age;
Student()          // Default Constructor
{
name = "Unknown";
age = 0;
}
void display()
{
System.out.println("Name: " + name + ", Age: " + age);
}
}
public class Main
{
public static void main(String[] args)
{
Student s = new Student();   // Default constructor is invoked
s.display();
}
}
```

C++

```
#include <iostream>
 using namespace std;
class Student
{
private:
string name;
int age;
public:
Student()     // Default Constructor
{
name = "Unknown";
age = 0;
}
void display()
{
cout << "Name: " << name << ", Age: " << age << endl; }
 };
int main()
{
 Student s;  // Default constructor is invoked
s.display();
return 0;  }
```

Python

```
class Student:
    def __init__(self):
        self.name = "Unknown"
        self.age = 0
    def display(self):
        print(f"Name: {self.name}, Age: {self.age}")
  s = Student()   # Default constructor is invoked
   s.display()
```

2. Parameterized Constructors

- A **parameterized constructor** is a constructor that takes arguments to initialize an object with specific values.
- It allows passing values as arguments when creating an object.
- Parameters are used within the constructor body to initialize the object's attributes.
- When using a parameterized constructor, initial values must be passed during object creation.
- Normal object declaration (e.g., Car c;) will not work if no default constructor is defined.
- Parameterized constructors can be called explicitly or implicitly:
  - **Explicit Call**: Car c = Car("Swift", "Red", 500);
  - **Implicit Call**: Car c("Swift", "Red", 500); (only in C++ but not in java and python).
- If a parameterized constructor is defined and no default constructor is provided explicitly, the compiler will not implicitly generate a default constructor. Creating an object without arguments (car c) will result in an error.
- **Java** does not support implicit calls to constructors. Objects must be created using the new keyword.
  Example: Car c = new Car("Swift", "Red", 500);.
- **In Java** it does not support default arguments in constructors. Instead, constructor overloading is used to achieve similar functionality.

- **C++** supports both explicit and implicit calls to parameterized constructors. It fully supports default arguments in constructors.
- **In Python** constructors are defined using __init__(). Parameterized constructors are the most common type in Python. Default arguments allow flexible object creation without needing multiple constructors.

- Whenever we define one or more non-default constructors(with parameters) for a class, a default constructor(without parameters) should also be explicitly defined as the compiler will not provide a default constructor in this case.
- It is used to initialize the various data elements of different objects with different values when they are created.
- It is used to overload constructors.
- We can also define default values for the arguments of parameterized constructors Just like normal functions. All the rules of the default arguments will be applied to these parameters.
- When the default values are assigned to every argument of the parameterized constructor, it is allowed to create the object without passing any parameters just like default constructors.
- This type of constructor works as both a default and parameterized constructor.

Note:- A default argument is a value provided in a function declaration that is automatically assigned by the compiler if the calling function doesn't provide a value for the argument. In case any value is passed, the default value is overridden.

Java	CPP	Python

class Car
{
String model;
String color;
int price;
// Default Constructor
Car()
{
this.model = "Unknown";
this.color = "Unknown";
this.price = 0;
}
// Parameterized Constructor
Car(String model, String color, int price)
{
this.model = model;
	#include <iostream>
	using namespace std;
	class Car
	{
	private:
	string model;
	string color;
	int price;
	public: **// Parameterized Constructor with Default Arguments**
	Car(string m = "Unknown", string c = "Unknown", int p = 0)
	{
	model = m;
	color = c;
	price = p;
		class Car:
		def __init__(self, model="Unknown", color="Unknown", price=0):
		self.model = model
		self.color = color
		self.price = price
		def display(self):
		print(f"Model: {self.model}, Color: {self.color}, Price: {self.price}")
		
		# Creating objects
		c1 = Car() **# Default arguments used**
		c2 = Car("Swift", "Red", 500) **# Explicit arguments override defaults**
		c1.display()
		c2.display()

```
this.color = color;
this.price = price;
}
void display()
{
System.out.println("Model: " + model +
", Color: " + color + ", Price: " + price);
}
}
public class Main
 {
public static void main(String[] args)
{
Car c1 = new Car(); // Default
constructor
Car c2 = new Car("Swift", "Red", 500);
// Parameterized constructor
c1.display();
c2.display();
} }
```

```
 }
void display()
{
cout << "Model: " << model << ", Color:
" << color << ", Price: " << price <<
endl;
}
};
 int main()
{
Car c1;  // Default arguments used Car
c2("Swift", "Red", 500);  // Explicit
arguments override defaults
c1.display();
c2.display();
return 0;
}
```

3. Copy constructor

- It is used to copy one object's value to another object of the same class.
- A copy constructor is used to initialize an object using another object of the same class.

In C++:

- If we don't define a copy constructor explicitly, the compiler creates one implicitly. Even if we define other constructors, the compiler will always create an implicit copy constructor unless we specifically provide one.
- This behaviour is unlike the default constructor, where if you define any constructor explicitly, the default constructor is deleted.

Syntax for explicit copy constructor:

It takes a reference to an object of the same class as an argument.

```
ClassName (ClassName &obj)
{
  // body_containing_logic
}
```

In Java:

- Java does not have explicit copy constructors. Instead, cloning is used.
- The clone() method from the Object class is used, and the class must implement the Cloneable interface.

In Python:

- Python does not have explicit copy constructors. Copying is handled using the copy module.
- Shallow copy (copy.copy()) and deep copy (copy.deepcopy()) are available.

Use of copy constructor:-

Constructs a new object by copying values from an existing object.
Can be used to perform deep copy.
Modify specific attributes during the copy process if needed.

Destructor

Destructors in C++:

- C++ supports destructors, and they are essential for managing dynamic memory and other resources.
- A destructor is a special member function used to destroy the objects created by constructor. It cleans up memory and release resources held by objects.
- A destructor has the same name as the class, but it is preceded by a tilde (~) symbol.
- Only one destructor is allowed per class.
- Destructors don't take any arguments and don't return any value therefore it cannot be overloaded
- Destructor cannot be declared as static and const;
- Destructor should be declared in the public section of the program.
- When an object goes out of scope or is explicitly deleted, its destructor is automatically called by the compiler.
- Destructors are executed in the reverse order of the constructors. This ensures that resources are released in the opposite order of their allocation.
- Like constructors, destructors can also be defined either inside or outside of the class.

The syntax for defining the destructor within the class

~ <class-name>(){}

The syntax for defining the destructor outside the class

<class-name>: : ~<class-name>() {}

Destructors in Java:

- Java does not have explicit destructors because it relies on garbage collection to automatically manage memory. However, a method called finalize() was historically used for cleanup, but it is now deprecated.
- Use try-with-resources or implement the AutoCloseable interface for resource cleanup.
- Explicit cleanup can be done using custom methods.

Destructors in Python:

- Python uses a special method __del__() to act as a destructor. However, since Python relies on garbage collection, destructors are rarely needed.
- __del__() is called when an object is deleted or goes out of scope.
- Use with caution as garbage collection may delay its execution.

Java

```
class Car {
   Car()
 {      System.out.println("Constructor
called");       }
   // Explicit cleanup method
   void cleanup() {
      System.out.println("Cleanup
called");
   }}
public class Main
{
   public static void main(String[] args)
{       Car c1 = new Car();
      c1.cleanup(); // Explicit cleanup
   }          }
```

CPP

```
#include <iostream>
using namespace std;

class Car {
public:
   Car() {
      cout << "Constructor called\n";
   }
   ~Car() {
      cout << "Destructor called\n";
   }   };
int main() {
   Car c1; // Constructor is called
   return 0; // Destructor is
automatically called          }
```

Python

```
class Car:
   def __init__(self):
      print("Constructor called")

   def __del__(self):
      print("Destructor called")

c1 = Car()  # Constructor is called
del c1      # Destructor is called
```

Subscribe Infeepedia youtube channel for computer science competitive exams
Download Infeepedia app and call or wapp on 8004391758

Access Specifiers

- To implement Data Hiding or encapsulation which is the most important concept of the oops, we use access specifiers or modifier.
- Access Specifiers in a class decide the accessibility of the class members(data and function) in other classes.
- Specifiers decide whether the class members will get directly accessed by the blocks present outside the class or not,
- By default, all members in a class are **private** in **C++** and **Java**, but in **Python**, they are **public** by default (with naming conventions to mimic private or protected behavior).
- In **Python**, access control is implemented using naming conventions:
  - _protectedMember: Protected
  - __privateMember: Private

There are three types of access modifiers in OOPS:

1. Public
2. Private
3. Protected

1. **Public:** If any class member (data members/member functions) is declared as public then:–
   1. They are accessible to everyone.
   2. They can be accessed using the direct member access operator (.).

2. **Private: If any class member is declared as private then:**
   - It can only be accessed by functions declared inside the class
   - It is not accessible from the outside of class
   - Only the member functions(in C++, jave ,python) or the friend functions/classes(only in C++) are allowed to access the private data members of a class.
   - If no specifier is mentioned then by default the class members are private (in C++ and java not in python).
   - In **Python**, private members are indicated using a double underscore prefix (__privateVar).
   - We can access the private data members of a class indirectly
     - ✓ By creating getter/setter functions
     - ✓ By initializing values via constructors
     - ✓ By setting values via public function

Java	CPP	Python

class Example
{
private
int privateVar;
// Setter
public void setVar(int value)
{ privateVar = value; }
// Getter
public int getVar()
{ return privateVar; } }
public class Main
{
public static void main(String[] args)
{
Example obj = new Example();
obj.setVar(10); **// Indirect access**
System.out.println(obj.getVar());
// Indirect access } }
	class Example
	{
	private: int privateVar;
	// Setter
	public: void setVar(int value)
	{ privateVar = value; }
	// Getter
	int getVar()
	{
	return privateVar;
	}
	};
	int main()
	{
	Example obj;
	obj.setVar(10); **// Indirect access**
	cout << obj.getVar(); **// Indirect access**
	}
		class Example:
		def __init__(self):
		self.__privateVar = 0 **# Private variable**
		
		# Setter
		def setVar(self, value):
		
		# Getter
		self.__privateVar = value
		def getVar(self):
		return self.__privateVar
		
		obj = Example()
		obj.setVar(10) **# Indirect access**
		print(obj.getVar()) **# Indirect access**

3. **Protected: If any class members are declared as protected then:-**
   - It is the same as private accessibility.But, derived (subclasses/child) classes can also access the protected class members.
   - The main difference between private and protected is that protected inheritance allows continued access to base class members whereas private inheritance prohibits the access of those members.
   - These protected members are inaccessible outside. But, are accessible in the derived class as protected data members.
   - **In C++,** the access level of protected members in the derived class depends on the inheritance mode:
     - Public Inheritance: Protected remains protected.
     - Protected Inheritance: Protected remains protected.
     - Private Inheritance: Protected becomes private.
   - **In Java,** protected members are accessible in the same package and through inheritance in other packages.
   - **In Python,** protected members are indicated with a single underscore (_protectedVar) and can be accessed in subclasses.

Note: This access through inheritance can alter the access modifier of the elements of base class in derived class depending on the mode of Inheritance.

Specifier	Within Same Class	In Derived Class	Outside the Class
Private	Yes	No	No
Protected	Yes	Yes	No
Public	Yes	Yes	Yes

Java

```
class Base
{
   protected int protectedVar;
}
class Derived extends Base
{
   public void setVar(int value)
{
protectedVar = value;
}
// Accessible in derived class
   public int getVar()
{
return protectedVar;
}
}
public class Main
{
   public static void main(String[] args)
{
Derived obj = new Derived();
obj.setVar(10); // Indirect access
System.out.println(obj.getVar());
}}
```

CPP

```
class Base
{
protected:
   int protectedVar;
};
class Derived : public Base {
 public:
   void setVar(int value)
 {
    protectedVar = value;
}
// Accessible in derived class
   int getVar()
{
return protectedVar;
 }   };
Int main()
{
Derived obj;
obj.setVar(10); // Indirect access
cout << obj.getVar();
}
```

Python

```
class Base:
   def __init__(self):
      self._protectedVar = 0  # Protected
variable

class Derived(Base):
   def setVar(self, value):
      self._protectedVar = value  #
Accessible in derived class

   def getVar(self):
      return self._protectedVar

obj = Derived()
obj.setVar(10)  # Indirect access
print(obj.getVar())
```

Encapsulation

- **Encapsulation** bundles data (variables) and methods (functions) into a single unit, called a class.
- It prevents direct access to data, allowing it to be accessed or modified only through specific methods.
- Works like a "locked box," where only authorized methods (keys) can access or change the data.
- Ensures data protection and hiding, keeping the code organized, secure, and easier to manage.
- **Implemented using access specifiers**: public, private, and protected.
- Improves **readability**, **maintainability**, and **security** by controlling how data is accessed or modified.
- Supports **read-only mode** (data can be accessed but not modified) and **write-only mode** (data can be modified but not accessed).



Java	CPP	Python

class Car
{
private String name;
private int mNumber;
public String color;
 	
// Method to set data
public void setData(String s1, int m1)
{ name = s1; mNumber = m1; }
 	
// Method to get data public
void getData() {
System.out.println("Name is: " + name);
System.out.println("m_number is: " + mNumber);
}
}
public class Main { public static void main(String[] args)
{
Car c = new Car();
c.color = "red";
c.setData("Alto", 102);
c.getData();
System.out.println("Color is: " + c.color); }
}
	class car {
	private:
	string name;
	int m_number;
	public:
	string color;
	void setdata(string s1,int m1)
	{
	name = s1;
	m_number = m1;
	}
	void getdata()
	{ cout<<"Name is: " <<name<<endl;
	
	cout<<"m_number is: "<<m_number<<endl;
	} };
	int main()
	{
	car c;
	c.color= “red”;
	c.setdata("Alto", 102);
	c.getdata();
	cout<<"Color is: " <<color<<endl;
	return 0;
	}
		class Car:
		def __init__(self):
		self.__name = None **# Private attribute**
		self.__m_number = None **# Private attribute**
		self.color = **None** **# Public attribute**
		
		# Method to set data
		def set_data(self, name, m_number):
		self.__name = name
		self.__m_number = m_number
		
		# Method to get data def
		get_data(self):
		print(f"Name is: {self.__name}")
		print(f"m_number is: {self.__m_number}")
		
		# Main program
		c = Car()
		c.color = "red"
		c.set_data("Alto", 102)
		c.get_data()
		print(f"Color is: {c.color}")

Types of Encapsulation

We can implement encapsulation in three ways:

1. Member variable encapsulation
2. Function encapsulation
3. Class encapsulation

1. Data Encapsulation

- Data members (variables) are declared private to restrict direct access.
- Access to these variables is provided through public getter and setter methods.
- Ensures controlled access and modification of data.

Subscribe Infeepedia youtube channel for computer science competitive exams
Download Infeepedia app and call or wapp on 8004391758

Example:

- C++/Java: Use private keyword and define getter/setter functions.
- Python: Use double underscore __ for private variables and define methods for access.

Java	CPP	Python

class Car
{
private String name;
// Setter method
public void setData(String s1)
{ name = s1; }
// Getter method
public void getData()
{ System.out.println("Name is: " +
name);
}
public static void main(String[] args)
{ Car c = new Car();
c.setData("Alto");
c.getData();
}
}
	#include<iostream>
	using namespace std;
	class car {
	private:
	string name;
	public:
	void setdata(string s1)
	{
	name=s1;
	}
	void getdata() {
	cout << "Name is: "<<name<<endl;
	} };
	int main() {
	car c;
	c.setdata("Alto");
	c.getdata(); return 0; }
		class Car:
		def __init__(self):
		self.__name = "" **# Private variable**
		
		# Setter method def
		set_data(self, name):
		self.__name = name
		
		# Getter method
		def get_data(self):
		print(f"Name is: {self.__name}")
		
		# Main function
		if __name__ == "__main__":
		c = Car()
		c.set_data("Alto")
		c.get_data()

2. Function Encapsulation

- Hides the internal logic of a method from the user.
- Only the required functionality is exposed via public methods.
- Internal details can be modified without affecting external code.
- **In Java and C++,** private methods are defined using the private keyword, and public methods are accessible outside the class.
- **In Python,** private methods are defined with a double underscore prefix (e.g., __speed_calculate), making them inaccessible directly outside the class.

Java	CPP	Python

class Car {
private void speedCalculate()
{
int speed;
if (mNumber > 200 && year > 2020)
{ speed = 150;
System.out.println("Speed is " +
speed); }
else {
speed = 100;
System.out.println("Speed is " +
speed); } }
public String name;
public int mNumber;
public int year;
public void getSpeed()
{ mNumber = 300;
year = 2021;
	using namespace std;
	class car {
	private:
	void speedcalculate() {
	int speed;
	if(m_number>200 && year>2020) {
	speed= 150;
	cout<< "speed is "<< speed<<endl;
	}
	else { speed=100;
	cout<< "speed is "<< speed<<endl; } }
	public:
	string name;
	int m_number;
	int year;
	void getspeed() {
	m_number=300;
	year=2021;
		class Car:
		def __init__(self):
		self.name = ""
		self.m_number = 0
		self.year = 0 # Private method
		def __speed_calculate(self):
		if self.m_number > 200 and self.year >
		2020:
		speed = 150
		print(f"Speed is {speed}")
		else:
		speed = 100
		print(f"Speed is {speed}")
		def get_speed(self):
		self.m_number = 300
		self.year = 2021
		self.__speed_calculate()
		# Main function

```
speedCalculate();    }
 public static void main(String[] args)
 { Car c = new Car();
c.getSpeed();
} }
```

```
    speedcalculate();   }   };
int main() {
  car c;
  c.getspeed();
   return 0;     }
```

```
 if __name__ == "__main__":
c = Car()
c.get_speed()
```

3. Class Encapsulation

- Combines data members and methods into a single unit (class).
- Prevents external interference and enforces data hiding.

Nested Class:

- In Java.C++,Python, a nested class is created using the class keyword inside another class.
- Nested classes in C++ can be made friend to the enclosing class or other classes for special access.

Access:

- In Java, the nested class instance is created using new and accessed directly.
- In Python, the nested class instance is created using self and accessed via methods.

Java

```
class Student {
   private class Address {
      private String street, city, state,
zipcode;
      public void getAddress() {
         street = "123 street";
         city = "Kanpur";
         state = "UP";
         zipcode = "211210";
         System.out.println("Address is " +
street + " " + city + " " + state + " " +
zipcode);
      }   }
   private Address a = new Address();

   public void myAddress() {
      a.getAddress();
   }
   public static void main(String[] args) {
      Student s = new Student();
      s.myAddress();
   }
}
```

Python

```
#include<iostream>
using namespace std;
class student {
private:
class Address {
  private:
    string street, city, state, zipcode;
public:
string getaddress()
    {
      street = "123 street";
      city = "Kanpur";
      state = "UP";
      zipcode = "211210";
cout<< "Address is "<<street <<" "<< city <<
" "
<< state <<" "<< zipcode << endl;
      return 0;     }   };
   Address a;
   public:
    void myaddress() {
      a.getaddress();
  }};
int main() {
  student s;
  s.myaddress();
  return 0;
}
```

C++

```
python
Copy code
class Student:
   class Address:
      def __init__(self):
         self.street = ""
         self.city = ""
         self.state = ""
         self.zipcode = ""

      def get_address(self):
         self.street = "123 street"
         self.city = "Kanpur"
         self.state = "UP"
         self.zipcode = "211210"
         print(f"Address is
{self.street} {self.city} {self.state}
{self.zipcode}")

   def __init__(self):
      self.a = self.Address()

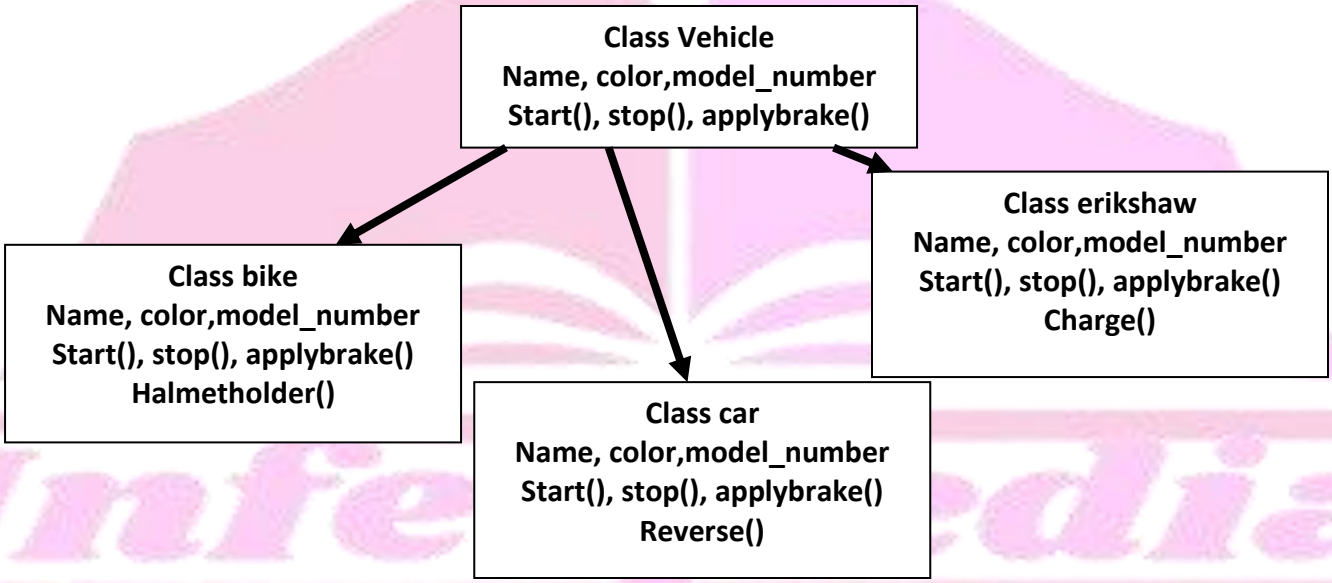
   def my_address(self):
      self.a.get_address()

if __name__ == "__main__":
   s = Student()
   s.my_address()
```

Inheritance

- In terms of programming it is a capability of class (child) to derive data and behaviour of another class (parent) which increases code reusability and reduce code size and redundancy.
- The class whose properties are acquired is called super parent/base class.
- The class which acquire all the properties is called derived /sub/child class.

Need of inheritance



Mode of inheritance

There are three modes of inheritance:

Base class Mode Of access specifiers	Inheritance mode		
	Public	Protected	Private
Public	Public	Protected	Private
Protected	Protected	Protected	Private
Private	Not accessible	Not accessible	Not accessible

Syntax of inheritance in Java, C++ and Python

Java	Cpp	Python

class BaseClass
{
// Base class members
}
class DerivedClass extends BaseClass
{
//Derived class members
}
	class BaseClass
	{
	// Base class members
	};
	class DerivedClass : public BaseClass
	{
	// Derived class members
	};
		class BaseClass:
		# Base class members
		pass
		
		class DerivedClass(BaseClass):
		# Derived class members
		pass

Aspect	C++	Java	Python
Keyword	: public	extends	Parent class in parentheses
Access Specifiers	public, protected, private	No explicit keyword; default is protected	No explicit keyword; uses naming conventions
Multiple Inheritance	Supported directly	Not supported (uses interfaces)	Supported directly
Code Style	Explicit and verbose	More concise than C++	Simplest and most concise

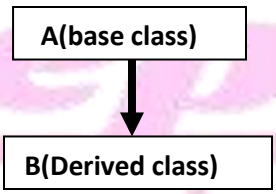
Types of Inheritance

There are five types of inheritance:

1. Single Inheritance
2. Multiple Inheritance
3. Multilevel Inheritance
4. Hierarchical Inheritance
5. Hybrid Inheritance

1. Single Inheritance

When the derived class inherits the properties of only one base class, it is called as Single Inheritance.
It is used to reuse, extend or modify the behavior of the parent class.




```
class Vehicle { String name; void start()
{
System.out.println("Vehicle started");
System.out.println("Name is " + name); }
void stop()
 {
System.out.println("Vehicle stopped");
} }
class Car extends Vehicle
{
int wheels;
 void reverse()
{
System.out.println("Car is in back gear.");
} }
public class Main
 {
public static void main(String[] args)
{
Car c = new Car();
c.name = "WagonR";
c.start();
c.stop();
c.reverse();
} }
```

```
#include<iostream>
using namespace std;
class vehicle
{   public:
string name;
void start()
{
   cout<< "vehicle started "<< endl;
   cout<< "name is " << name<< endl;
 }
void stop()
{   cout<< "vehicle stopped " << endl;
} };
class car: public vehicle
{
public:
int wheels;
void reverse()
{   cout<< "car is in back gear. "<< endl;
} };
int main()
{   car c;
   c.name="WagonR";
   c.start();
   c.stop();
   c.reverse();  }
```

```
class Vehicle:
def __init__(self):
self.name = ""

def start(self):
print("Vehicle started")
print(f"Name is {self.name}")
def stop(self):
print("Vehicle stopped")

class Car(Vehicle):
def __init__(self):
super().__init__()
self.wheels = 0

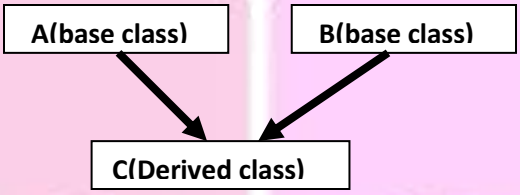
def reverse(self):
print("Car is in back gear.")
c = Car()
c.name = "WagonR"
c.start()
c.stop()
c.reverse()
```

Output:-

vehicle started
name is WagonR
vehicle stopped
car is in back gear.

Multiple Inheritance

When a derived /child/sub class inherits the properties of more than one base/parent/super class, it is called multiple inheritance.



Java

```
interface Electronics
{
void on();
void off();
}
interface Mechanical
{
void pushed();
}
class Car implements Electronics,
Mechanical
 {
String range;
String weight;
int wheels;
public void on()
{
System.out.println("On button is
pressed");
System.out.println("Access range is " +
range);
}
public void off()
 {
System.out.println("Off button is
pressed");
}
public void pushed()
{
System.out.println("Push button is
pressed");
System.out.println("Weight is " +
weight);
}
public void reverse()
{
```

CPP

```
#include<iostream>
using namespace std;
class Electronics {
public:
string range;
void on()
{
   cout<< "on button is pressed "<<
endl;
   cout<< "access range is " << range<<
endl;
 }
void off()
{
   cout<< "off button is pressed" <<
endl;
}  };
class mechanical
{
   public:
string weight;
void pushed()
{
   cout<< " push button is pressed "<<
endl;
   cout<< "weight is " << weight<< endl;
 }  };

class car: public Electronics, public
mechanical
{
public:
int wheels;
void reverse()
{
   cout<< "car is in back gear. "<< endl;
} };
```

Python

```
class Electronics:
def __init__(self):
self.range = None

def on(self):
print("On button is pressed")
print(f"Access range is {self.range}")

def off(self):
print("Off button is pressed")

class Mechanical:
def __init__(self):
self.weight = None

def pushed(self):
print("Push button is pressed")
print(f"Weight is {self.weight}")

class Car(Electronics, Mechanical):
def __init__(self):
Electronics.__init__(self)
Mechanical.__init__(self)
self.wheels = None

def reverse(self):
print("Car is in back gear.")

# Main program
c = Car()
c.range = "10 mtr"
c.weight = "300kg"
c.on()
c.pushed()
c.reverse()
```

```
System.out.println("Car is in back gear.");
}
public static void main(String[] args)
{
Car c = new Car();
c.range = "10 mtr";
c.weight = "300kg";
c.on();
c.pushed();
c.reverse();
}
}
```

```
int main()
{
   car c;
   c.range="10 mtr";
   c.weight= "300kg";
   c.on();
   c.pushed();
   c.reverse();
 }
```

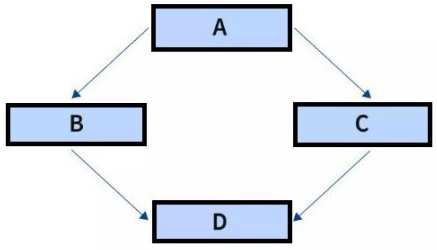
Output:-

On button is pressed
Access range is 10 mtr
Push button is pressed
Weight is 300kg
car is in back gear.

Feature	C++	Java	Python
Multiple Inheritance	Supported directly	Supported via interfaces	Supported directly
Syntax	class Car : public Class1, public Class2	class Car implements Interface1, Interface2	class Car(Class1, Class2)
Ease of Use	Verbose	Requires interface implementation	Simplest and direct
Initialization	Implicit for parent classes	Explicit in interfaces	Explicit for all parent classes

Diamond Problem

- Suppose A's properties are inherited by B and C classes and then D inherits the properties of both B and C, as this situation is creating a diamond shape, therefore, this problem is known as "diamond problem".
- In this situation classes B and C both can access the properties of A.
- D can access the properties of B and C. So D can access A via B and via C.
- This causes ambiguity. As the compiler will get confused.
- In **C++**, the diamond problem is handled using **virtual inheritance** to avoid ambiguity.
- In **Java**, the diamond problem does not occur because **multiple inheritance with classes is not allowed**. Instead, Java uses **interfaces** to achieve multiple inheritance, which avoids ambiguity since interfaces do not have implementations.

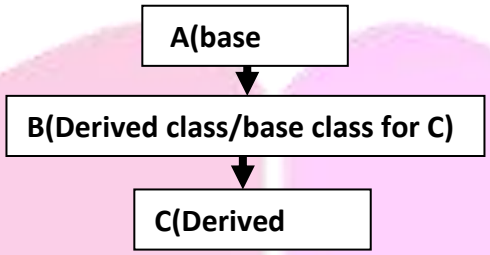


- In **Python**, the diamond problem is resolved using the **Method Resolution Order (MRO)**, which follows the **C3 Linearization Algorithm**. Python ensures that each class in the hierarchy is called only once.

Feature	C++	Java	Python
Occurrence	Yes, in multiple inheritance	No, as Java does not support multiple inheritance with classes	Yes, but resolved using MRO
Resolution	Virtual inheritance	Interfaces (no implementation conflict)	C3 Linearization (MRO)
Ambiguity	Can occur without virtual inheritance	Not possible	Automatically resolved
Ease of Handling	Requires explicit virtual inheritance	No special handling required	No special handling required

Multilevel Inheritance

When a derived class inherits the properties of the base class and further it become the base class(parent class) to the other class (or further it's property is inherited by some other class), it is called Multilevel Inheritance.
There can be any number of levels i.e any number of derived classes in multilevel inheritance. The chain of inheritance can extend to multiple levels, allowing classes to inherit properties and behaviors from their ancestors.



Java

```
class Vehicle
{
 String name;
 void start()
 {
System.out.println("Vehicle started");
System.out.println("Name is " +
name);
 }
void stop()
{
System.out.println("Vehicle stopped");
} }
class Car extends Vehicle
{
int wheels;
void reverse()
{
System.out.println("Car is in back
gear."); } }
class MarutiSuzuki extends Car
{
int mNumber;
void print(int x)
{ mNumber = x;
System.out.println("Model number is
" + mNumber);
 } }
public class Main {
public static void main(String[] args)
{
MarutiSuzuki m = new MarutiSuzuki();
m.name = "WagonR";
m.start();
m.stop();
m.reverse();
m.print(193);
} }
```

CPP

```
class vehicle
{
public:
string name;
void start()
{
   cout<< "vehicle started "<< endl;
   cout<< "name is " << name<< endl;
 }
void stop()
{
   cout<< "vehicle stopped " << endl;
} };
class car: public vehicle
{
public:
int wheels;
void reverse()
{
   cout<< "car is in back gear. "<< endl;
} };
class maruti_suzuki: public car
{    public:
   int m_number;
   void print(int x)
   {   m_number =x;
      cout<< "model number is "
<<m_number<<endl;
   }
  };
int main()
{
   maruti_suzuki m;
   m.name="WagonR";
   m.start();
   m.stop();
   m.reverse();
   m.print(193);        }
```

Python

```
class Vehicle:
def __init__(self):
self.name = ""

def start(self):
print("Vehicle started")
print(f"Name is {self.name}")

def stop(self):
print("Vehicle stopped")
class Car(Vehicle):

def __init__(self):
super().__init__()
self.wheels = 0

def reverse(self):
print("Car is in back gear.")
class MarutiSuzuki(Car):

def __init__(self):
super().__init__()
self.m_number = 0

def print_model(self, x):
self.m_number = x
print(f"Model number is
{self.m_number}")

# Main code
m = MarutiSuzuki()
m.name = "WagonR"
m.start()
m.stop()
 m.reverse()
m.print_model(193)
```

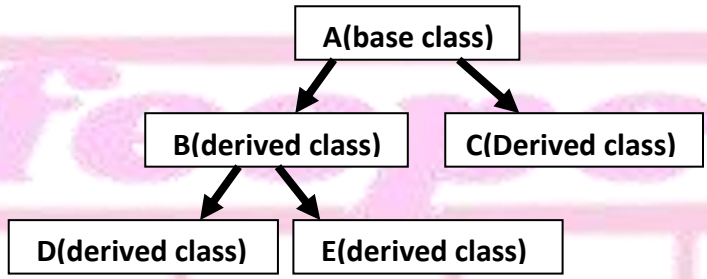
Output:-

Vehicle started
Name is WagonR
Vehicle stopped
car is in back gear.
Model number is 193.

Aspect	C++	Java	Python
Inheritance Syntax	class Derived : public Base	class Derived extends Base	class Derived(Base)
Access Specifiers	Public, Private, Protected	Public, Private, Protected (default for members is package-private)	No explicit specifiers; everything is public by default
Constructors	Must be explicitly defined and called	Uses super() to call parent constructor	Uses super().__init__() to call parent constructor

Hierarchical Inheritance

When more than one derived class inherit the properties of single base class, it is called Hierarchical Inheritance.



Java

```
class Vehicle {
   String name;
   void start()
{
      System.out.println(name + " is
started");
   }
   void stop() {
      System.out.println(name + " is
stopped");
   }
}
class Car extends Vehicle {
   int wheels;

   void reverse() {
      System.out.println("Car is in back
gear.");
   }
}
class Bike extends Vehicle {
   String type;
   void helmetHolder() {
      System.out.println("This is a " +
type + " bike");
```

C++

```
class vehicle
{
public:
string name;
void start()
{
   cout<< name <<" is started "<< endl;
 }
void stop()
{
   cout<< name <<" is stopped "<< endl;
} };
class car: public vehicle
{ public:
int wheels;
void reverse()
{ out<<"car is in back gear."<< endl;
} };
class bike: public vehicle
{   public:
   string type;
 void halmetholder()
{
cout<< "this is a "<< type<< "bike"
<<endl;
```

Python

```
class Vehicle:
   def __init__(self, name=""):
      self.name = name

   def start(self):
      print(f"{self.name} is started")

   def stop(self):
      print(f"{self.name} is stopped")


class Car(Vehicle):
   def __init__(self, name="",
wheels=4):
      super().__init__(name)
      self.wheels = wheels

   def reverse(self):
      print("Car is in back gear.")

class Bike(Vehicle):
   def __init__(self, name="",
bike_type=""):
      super().__init__(name)
      self.type = bike_type
```

```
    System.out.println("Helmet holder is opened");
  }
}
public class Main {
  public static void main(String[] args) {
    Car c = new Car();
    Bike b = new Bike();
    c.name = "WagonR";
    b.name = "Pulsar";
    c.start();
    b.stop();
    c.reverse();
    b.type = "sports";
    b.helmetHolder();
  }
}
```

```
 cout<< "halmet holder is opened" << endl;
  } };
int main()
{
  car c;
  bike b;
  c.name="WagonR";
  b.name="pulser";
  c.start();
  b.stop();
  c.reverse();
}
```

```
  def helmet_holder(self):
    print(f"This is a {self.type} bike")
    print("Helmet holder is opened")


# Main Code
c = Car(name="WagonR")
b = Bike(name="Pulsar", bike_type="sports")

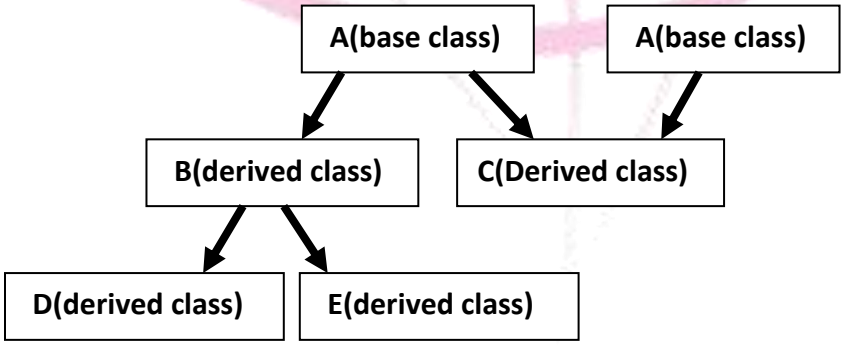
c.start()
b.stop()
c.reverse()
b.helmet_holder()
```

Output:-
WagonR is started
pulser is stopped
car is in back gear.

Aspect	C++	Java	Python
Constructor	Not mandatory, but can initialize manually	Uses default or parameterized constructors	Uses __init__ method for initialization
Access Modifiers	Public, Private, Protected	Public, Protected, Private (default: package-private)	No explicit access modifiers; attributes are public by default
Inheritance Syntax	class Derived : public Base	class Derived extends Base	class Derived(Base)

Hybrid inheritance

It is a combination of one or more types of inheritance.



```
class World
{
 World()
{ System.out.println("We are in
World!");
} }

 // Single Inheritance
class Continent extends World
{
Continent()
{
System.out.println("We are in
Continent");
} }

class Country
{
Country()
{
System.out.println("We are in
Country");
} }

// Multiple Inheritance Simulation
using Interfaces
class India extends Continent
 { Country country = new Country();
// Simulate multiple inheritance
India()
{ System.out.println("We are in
India!"); } }
public class Main
{
public static void main(String[] args)
{
India I = new India();
}
 }
```

```
#include<iostream>
using namespace std;
class World {
  public:
    World() {
      cout << "we are in World!"<< endl;
    } };
// Single Inheritance.
class Continent: public World {
  public:
    Continent() {
      cout << "We are in Continent\n";
    }
};
class Country {
  public:
    Country() {
      cout << "We are in Country"<<endl;
    } };
// multiple Inheritance.
class India: public Continent, public
Country {
  public:
    India() {
      cout << "We are in India!";
    } };
int main() {
  India I;
  }
```

```
class World:
def __init__(self):
print("We are in World!")

# Single Inheritance
class Continent(World):
def __init__(self):
 super().__init__()
print("We are in Continent")
class Country:
def __init__(self):
print("We are in Country")

# Multiple Inheritance
class India(Continent, Country):
def __init__(self):
Continent.__init__(self)

# Explicitly call Continent's constructor
Country.__init__(self)

# Explicitly call Country's constructor
print("We are in India!")

# Main Code
 I = India()
```

Output:-
we are in World!
We are in Continent
We are in Country
We are in India!

Subscribe Infeepedia youtube channel for computer science competitive exams
Download Infeepedia app and call or wapp on 8004391758

Polymorphism

Polymorphism = "poly" + "morphs" = many forms.
In real life a person can play different role in like a lady can be mom, daughter, wife, teacher etc.

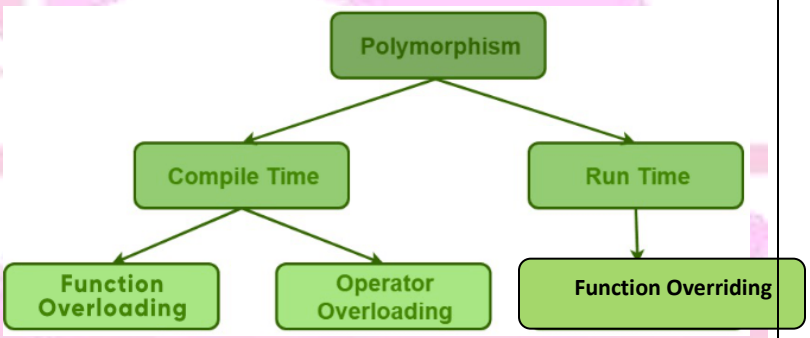
Types of Polymorphism

1. Compile time polymorphism
   a) Function Overloading
   b) Operator Overloading(not in java)

2. Run time polymorphism
   a) Function overriding

Compile time polymorphism

- This type of polymorphism is achieved by function overloading or operator overloading (only in C++).
- Information about overloading (type, argument etc) is known prior so we can check it at the time of program compilation.
- Here compiler selects the appropriate function to run at compile time.
- It is also known as static binding or early binding.
- In Java only supports **method overloading** (no operator overloading).
- In Python function overloading does not directly supported but simulates it using **default arguments** and operator overloading is supported via **magic methods**.



Method Overloading

- In Method overloading more than one method is having the same name but with the different number of parameters or the type of the parameters or order of the parameters.
- The **compiler** determines which function to call based on the **number, type, and order** of arguments passed during the function call.
- **In Python not directly supported** like in C++ or Java. But achieved using **Default arguments** and **Variable-length arguments**.

Example:

```
class Calculator
{
// Two integers
public: int add(int a, int b)
 { return a + b; }
// Two doubles
double add(double a, double b)
 {
return a + b;
 }
// Three integers
int add(int a, int b, int c)
{ return a + b + c; }
 };
```

```
class Calculator
{
// Two integers
 int add(int a, int b)
{ return a + b; }

// Two doubles
double add(double a, double b)
{ return a + b; }

// Three integers
int add(int a, int b, int c)
 { return a + b + c; }
}
```

```
class Calculator:
def add(self, a, b, c=0):
return a + b + c
# Handles two or three arguments
```

Feature	C++	Java	Python
Function Overloading	Fully supported	Fully supported	Simulated using defaults or *args
Key Differentiators	Number, type, order of params	Number, type, order of params	Default or variable-length params
Compile-Time Binding	Yes	Yes	No, runtime evaluation

Operator Overloading

- By using operator overloading we can play with operators.
- It allows us to create operators that can serve specific purposes with user-defined functions.
- In operator overloading, different operators display different implementations based on their parameters or signatures.
- Suppose + operator work as the minus one.
- We use keyword "operator" (in C++ only) along with the operator symbol that we want to overload.
- These operator functions get invoked when their corresponding operators are used in the program.
- We can overload almost all operators except

Operators That Can Be Overloaded:

- Arithmetic operators : - , + , / , * , % and -=, +=, /= , *= , %=
- Relational Operators : !=, ==, > ,< ,>=, <=
- Bitwise operators : &, |, ^ ,<< ,>> and |=, &= ,>>=, <<=, ^=
- Logical operators: && ,||
- Memory management : new[], new, delete[], delete

Operators That Cannot Be Overloaded:

.(dot operator), :: (scope operator), ?: (ternary operator), size of operator, ".*" and typeid.

Feature	C++	Java	Python
Operator Overloading	Fully supported using operator	Not directly supported	Supported via dunder methods
Syntax	ClassName operator+	Use method like add()	Define __add__, __sub__, etc.
Flexibility	High	Simulated through methods	High
Ease of Use	Moderate	Easy	Easy

C++: Templates

- C++ templates allow the creation of generic functions and classes that can work with any data type.
- The template keyword is used to define templates.
- **Function Templates**: Create functions that work with any data type.
- **Class Templates**: Create classes that work with any data type.
- They allow for compile-time instantiation of code based on different data types, facilitating compile-time polymorphism by generating specific code for each data type at compile time.

Syntax:

```
template <typename T>
T functionName(T param) {
   // Function body
}
```

Java: Generics

- Java uses generics to achieve the same functionality as templates.
- Generics ensure type safety and are checked at compile time.
- Generics can be used with classes, methods, and interfaces.
- Java generics are implemented using type erasure, meaning type information is erased at runtime.

Syntax:

```
class ClassName<T> {
   // Generic class body
}
public <T> T methodName(T param) {
   // Generic method body
}
```

Python: Generics

- Python achieves generic programming through duck typing and the typing module.
- Python is dynamically typed, so generics are not enforced at runtime but are used for type hinting and static type checking.
- The typing module provides support for generics using TypeVar and generic classes.
- Python functions and classes are inherently generic due to its dynamic nature.

Syntax:

```
from typing import TypeVar, Generic
T = TypeVar('T')
class ClassName(Generic[T]):
   # Generic class body
```

Runtime Polymorphism

- Run time polymorphism is achieved by method overriding where two or more functions with the same name, arguments, and return type base and derived classes. The base class is called overridden.
- This method has a comparatively slower execution rate than compiletime polymorphism since all the methods that need to be executed are called during run time.
- It is also called Late binding and dynamic polymorphism.
- Runtime polymorphism occurs when a function in a base class is overridden in a derived class, and the appropriate function is called at runtime based on the object's type.
- The base class function is marked as virtual in C++ or overridden using the @Override annotation in Java.
- In Python, method overriding is directly supported without additional keywords.
- A base class pointer/reference is used to call the derived class function at runtime.
- In C++ Use the virtual keyword in the base class for the function to allow overriding.

Function overriding

Java	CPP	Python

class Vehicle
{
void start()
{
System.out.println("Vehicle started");
} }
class Car extends Vehicle
{
@Override
void start()
	#include <iostream> using namespace std;
	class Vehicle
	{ public:
	virtual void start()
	{
	cout << "Vehicle started" << endl;
	} };
	class Car : public Vehicle
	{
		class Vehicle:
		
		def start(self):
		print("Vehicle started")
		
		class Car(Vehicle):
		def start(self):
		print("Car started")
		
		v = Car() # Derived class object

{
System.out.println("Car started");
} }
public class Main { public static void main(String[] args)
{
Vehicle v = new Car();
// Base class reference to derived class object
v.start();
// Calls Car's start() at runtime
} }
	public: void start() override
	{
	cout << "Car started" << endl; } };
	int main()
	{ Vehicle* v = new Car();
	// Base class pointer to derived class object
	v->start();
	// Calls Car's start() at runtime
	delete v;
	return 0;
	}
		v.start() # Calls Car's start() at runtime

Comparison between C++, Java and Python

Feature	C++	Java	Python
Keyword for Overriding	virtual and override	@Override annotation	No special keyword needed
Function Resolution	Virtual table (vtable)	Dynamic method dispatch	Direct method overriding
Ease of Use	Moderate	Easy	Very easy
Base Class Reference	Required for polymorphism	Required for polymorphism	Optional

Normal Programs with function overriding

Java	CPP	Python

class Car {
public void display() {
System.out.println("Basic car has manual functions");
} }
class BMW extends Car {
@Override
public void display() {
System.out.println("BMW has automatic functions");
} }
public class Main {
public static void main(String[] args) {
Car c = new Car();
BMW b = new BMW();
c.display(); // Calls the display method of the Car class
b.display(); // Calls the overridden display method of the BMW class
}
}
	#include <iostream>
	using namespace std;
	class car
	{
	public:
	void display()
	{
	cout << "Basic car has manual functions" <<endl;
	}
	};
	class BMW: public car
	{
	public:
	void display()
	{ cout << "BMW has automatic functions" <<endl;
	} };
	int main()
	{ car c;
	BMW b;
	c.display();
	b.display(); }
		class Car:
		def display(self):
		print("Basic car has manual functions")
		
		class BMW(Car):
		def display(self):
		print("BMW has automatic functions")
		
		# Main function
		if __name__ == "__main__":
		c = Car()
		b = BMW()
		c.display() # Calls the display method of Car
		b.display() # Calls the overridden display method of BMW

Output:

Basic car has manual functions
BMW has automatic functions

Program of function overriding with Base class reference pointing to a derived class object

```
class Car {
   public void display() {
      System.out.println("Basic car has manual functions");
   }
}
class BMW extends Car {
   @Override
   public void display() {
      System.out.println("BMW has automatic functions");
   }  }
public class Main {
   public static void main(String[] args) {
  Car c = new BMW();   // Polymorphism: Base class reference pointing to a derived class object
   c.display();      // Calls the overridden method in the BMW class
   }  }
```

```
#include <iostream>
using namespace std;
class car
{
 public:
 void display()
 {
 cout << "Basic car has manual functions"<<endl;
 } };
class BMW: public car
{
 public:
 void display()
 {
 cout << "BMW has automatic functions" <<endl;    } };
int main()
{   car *c = new BMW();
 c->display();     }
```

```
class Car:
   def display(self):
      print("Basic car has manual functions")

class BMW(Car):
   def display(self):
      print("BMW has automatic functions")

# Main function
if __name__ == "__main__":
   c = BMW()  # Creating an instance of BMW and assigning it to a variable of type Car
   c.display()  # Calls the overridden display method of BMW
```

Output:
BMW has automatic functions

Program of overridden method in the derived class

Java

```
class Car {
 public void print()
{   System.out.println("This is a basic car");    }
public void show()
{
System.out.println("Manual car features");
} }
class BMW extends Car
{
@Override
public void print()
{
System.out.println("This is an advanced car");
}
@Override
public void show()
{ System.out.println("Automatic car features"); } }
public class Main
 {
public static void main(String[] args)
 {
Car c= new Car();   // Base class
```

CPP

```
#include <iostream>
using namespace std;
class car {
public:
 void print()
{ cout << "This is a basic car "<<endl; }
 virtual void show()
{ cout << "Mannual car features"<<endl; }
};
class BMW : public car {
public:
 void print()
{ cout << "This is an advance car"<<endl; }
 void show()
{
cout << "Automatic car features"<<endl; }
};
int main()
{
 car* c;
 BMW b;
 c = &b;
 c->print();
 c->show();
 return 0;
}
```

Python

```
class Car:
   def print(self):
      print("This is a basic car")

   def show(self):
      print("Manual car features")

class BMW(Car):
   def print(self):
      print("This is an advanced car")

   def show(self):
      print("Automatic car features")

# Main function
if __name__ == "__main__":
   c = BMW()  # Creating an instance of BMW
   c.print()  # Calls the overridden print method in BMW
   c.show()   # Calls the overridden show method in BMW
```

```
reference
BMW b = new BMW();  // Derived class object
c.print();   // Calls the `print` method of the base class (no overriding here)

c = b;  // Base class reference pointing to derived class object
c.show();   // Calls the overridden `show` method in the derived class (runtime polymorphism)
 } }
```

Output:

This is a basic car
Automatic car features

Difference Between Compile-Time Polymorphism and Run-Time Polymorphism

Aspect	Compile-Time Polymorphism	Run-Time Polymorphism
Definition	The method call is resolved at compile time.	The method call is resolved at runtime.
Other Names	Static binding, early binding, overloading.	Dynamic binding, late binding, overriding.
Inheritance	Inheritance is not required.	Inheritance is required.
Implementation	Achieved through **function overloading** and **operator overloading**.	Achieved through **method overriding** and **virtual functions**.
Execution Speed	Faster, as the method to execute is determined during compilation.	Slower, as the method to execute is determined at runtime.
Flexibility	Less flexible because decisions are made at compile time.	More flexible because decisions are made at runtime.
Method Signature	Methods with the **same name** but **different parameters** or **return types**.	Methods with the **same name and parameters** in different classes.
Examples in C++	Function Overloading Operator Overloading	Virtual Functions Pointers to Base Class
Examples in Java	Method Overloading	Method Overriding Using @Override Annotation
Examples in Python	Default Arguments Method Overloading (limited support)	Method Overriding

Virtual function

- It is supported in C++.
- It is declared using 'virtual' keyword.
- It is used to achieve function overriding in run-time polymorphism.
- When a base class pointer object pointing to derived class object but still access base class methods then we use virtual keyword to resolve this conflict.
- No matter what kind of pointer or reference is used for the function Call virtual functions ensure that the right function is called for an object.
- Virtual functions cannot be static.
- Virtual functions should have the same prototype in the base class and any derivations.
- A virtual function can be a friend function of another class.
- Virtual functions should be accessed using a pointer or reference of base class type to achieve runtime polymorphism.
- A class may have a virtual destructor but it cannot have a virtual constructor.

Syntax:

```
class Base {
public:
  virtual void show() {
    cout << "Base class show()" << endl;
  } };
class Derived : public Base {
public:
  void show() override { // Override base class method
    cout << "Derived class show()" << endl;
  } };
```

Java: Virtual Functions

- In Java, all **non-static methods** are **virtual by default**.
- The final keyword can be used to prevent overriding, making a method non-virtual.
- Polymorphism in Java is achieved through method overriding.
- Methods are resolved at runtime based on the actual object type (dynamic method dispatch).

Syntax:

```
class Base {
  public void show() {
    System.out.println("Base class show()");
  } }
class Derived extends Base {
  @Override
  public void show() {
    System.out.println("Derived class show()");
  } }
```

Python: Virtual Functions

- In Python, **all methods** are virtual by default.
- Overriding is achieved by defining a method in the derived class with the same name as in the base class.
- Polymorphism is inherent in Python because of its dynamic nature.
- Methods are resolved at runtime based on the actual object type.

Syntax:

```
class Base:
  def show(self):
    print("Base class show()")
class Derived(Base):
  def show(self):
    print("Derived class show()")
```

Java	CPP	Python

class Vehicle {
public void start() {
System.out.println("Vehicle started");
} }
class Car extends Vehicle {
@Override
public void start() {
System.out.println("Car
	#include <iostream>
	using namespace std;
	class Vehicle
	{ public:
	virtual void start()
	{ cout << "Vehicle started" << endl;
	} };
	class Car : public Vehicle
	{ public: void start() override {
		class Vehicle:
		def start(self):
		print("Vehicle Started")
		
		class car(Vehicle):
		def start(self):
		print("Car Started")
		
		# Main Program

```
started");
   } }
public class Main {
   public static void main(String[]
args) {
      Vehicle v = new Car();  // Base
class reference to derived class
object
      v.start();    // Calls Car's start() at
runtime
   }  }
```

```
cout << "Car started" << endl; } };
int main()
{ Vehicle* v = new Car();
// Base class pointer to derived class object
v->start();
// Calls Car's start() at runtime
delete v;
return 0;
 }
```

```
Vehicle_ref = car()  # Base class
reference to Derived object
Vehicle_ref.start()      # Output:
Derived class show()
```

Output:
Car started

Comparison of Virtual Functions in C++, Java and Python

Feature	C++	Java	Python
Default Behavior	Non-virtual by default	Virtual by default	Virtual by default
Keyword	virtual	No keyword (all non-static are virtual)	No keyword required
Override Enforcement	Optional (override keyword is optional)	Recommended (@Override annotation)	Implicitly overridden
Final Methods	Use final keyword to prevent override	Use final keyword	No direct equivalent (not override by convention)
Polymorphism	Requires virtual keyword	Achieved via overriding	Achieved via overriding
Method Resolution	Determined at runtime (vtable)	Determined at runtime (dynamic dispatch)	Determined at runtime (dynamic typing)

In C++: Pure Virtual Functions

- A pure virtual function is a function that must be implemented by derived classes, making the base class abstract.
- The implementation is deferred to the derived class, ensuring that the derived classes provide their own specific behavior for that function.
- A pure virtual function is declared in a base class by assigning = 0 to the function declaration.
- A class containing at least one pure virtual function becomes an abstract class and cannot be instantiated.
- The derived class must implement the pure virtual function; otherwise, it will also become abstract.
- Abstract classes cannot be instantiated directly.

Syntax:

```
class Base {
public:
   virtual void function() = 0;    // Pure virtual function
};
```

Java: Abstract Methods

- In Java, a pure virtual function is called an abstract method.
- Declared using the abstract keyword in an abstract class.
- Abstract methods have no body and must be implemented by derived (sub)classes.
- A class containing at least one abstract method must be declared as abstract.
- Abstract classes cannot be instantiated.
- Subclasses must provide concrete implementations for all abstract methods.

Syntax:

```
abstract class Base {
   abstract void show(); // Abstract method
}
```

Python: Abstract Methods

- In Python, abstract methods are defined in a base class using the @abstractmethod decorator from the abc (Abstract Base Class) module.
- A class containing abstract methods becomes an abstract class and cannot be instantiated.
- The ABC module is used to create abstract base classes.
- Abstract methods must be implemented in derived classes.
- Attempting to instantiate an abstract class will result in a TypeError.

Syntax:

```
from abc import ABC, abstractmethod
class Base(ABC):
@abstractmethod
def show(self):
pass
```

Java	CPP	Python

abstract class Shape {
  abstract void draw(); // Abstract method
}
class Circle extends Shape {
  @Override
  void draw() {
  System.out.println("Drawing Circle");
  }
}
public class Main {
  public static void main(String[] args) {
  Shape shape = new Circle();
  shape.draw(); // Output: Drawing Circle
  }
}
	#include <iostream>
	using namespace std;
	class Shape {
	public:
	  virtual void draw() = 0; // Pure virtual function
	};
	class Circle : public Shape {
	public:
	  void draw() override {
	  cout << "Drawing Circle" << endl;
	  }
	};
	int main() {
	  Shape* shape = new Circle();
	  shape->draw(); // Output: Drawing Circle
	  delete shape;
	  return 0;
	}
		from abc import ABC, abstractmethod
		
		class Shape(ABC):
		  @abstractmethod
		  def draw(self):
		  pass
		
		class Circle(Shape):
		  def draw(self):
		  print("Drawing Circle")
		
		# Main Program
		shape = Circle()
		shape.draw() # Output: Drawing Circle

Comparison of Pure Virtual Functions in C++, Java and Python

Feature	C++	Java	Python
Keyword	= 0	abstract	@abstractmethod
Base Class	Abstract if it has pure virtual	Abstract if it has abstract	Abstract if it has abstract
Inheritance	Derived classes must implement	Subclasses must implement	Subclasses must implement
Instantiation	Not allowed for abstract class	Not allowed for abstract class	Not allowed for abstract class
Modules/Headers	No special header needed	No special imports	Requires abc module

Subscribe Infeepedia youtube channel for computer science competitive exams
Download Infeepedia app and call or wapp on 8004391758

Abstract Class

An abstract class serves as a blueprint for other classes. It cannot be instantiated directly and is used to define common behaviors that must be implemented by derived or child classes.

In C++: Abstract Class

- An abstract class is a class that contains at least one pure virtual function.
- A pure virtual function is declared by assigning = 0 to the function declaration.
- Abstract classes cannot be instantiated.
- Derived classes must implement all pure virtual functions, or they will also become abstract.
- Abstract classes can have constructors, destructors, and concrete (non-pure virtual) functions.

Syntax:

```
class AbstractClass {
public:
   virtual void abstractMethod() = 0; // Pure virtual function
   void concreteMethod() {
      cout << "Concrete method in abstract class" << endl;
   }
};
```

In Java: Abstract Class

- An abstract class is declared using the abstract keyword.
- It can contain both abstract methods (without a body) and concrete methods (with a body).
- Abstract classes cannot be instantiated.
- Subclasses must provide implementations for all abstract methods unless the subclass is also abstract.
- Abstract classes can have constructors, which are called when a subclass object is created.

Syntax:

```
abstract class AbstractClass {
   abstract void abstractMethod(); // Abstract method
   void concreteMethod() {
      System.out.println("Concrete method in abstract class");
   }
}
```

In Python: Abstract Class

- Abstract classes in Python are defined using the ABC (Abstract Base Class) module.
- Methods are marked as abstract using the @abstractmethod decorator.
- Abstract classes cannot be instantiated.
- Subclasses must implement all abstract methods.
- Abstract classes can have concrete methods as well.

Syntax:

```
from abc import ABC, abstractmethod

class AbstractClass(ABC):
   @abstractmethod
   def abstract_method(self):
      pass
   def concrete_method(self):
      print("Concrete method in abstract class")
```

Java	CPP	Python

abstract class Shape {
  abstract void draw(); // Abstract method
}
 	
class Circle extends Shape {
  @Override
  void draw() {
 	
System.out.println("Drawing Circle");
  }
}
public class Main {
  public static void main(String[] args) {
  Shape shape = new Circle();
  shape.draw(); // Output: Drawing Circle
  }
}
	#include <iostream>
	using namespace std;
	
	class Shape {
	public:
	  virtual void draw() = 0; // Pure virtual function
	};
	
	class Circle : public Shape {
	public:
	  void draw() override {
	  cout << "Drawing Circle" << endl;
	  }
	};
	int main() {
	  Shape* shape = new Circle();
	  shape->draw(); // Output: Drawing Circle
	  delete shape;
	  return 0;
	}
		from abc import ABC, abstractmethod
		
		class Shape(ABC):
		  @abstractmethod
		  def draw(self):
		  pass
		
		class Circle(Shape):
		  def draw(self):
		  print("Drawing Circle")
		
		# Main Program
		shape = Circle()
		shape.draw() # Output: Drawing Circle

Comparison of Abstract Classes

Feature	C++	Java	Python
Definition	Contains at least one pure virtual function	Declared with abstract keyword	Uses ABC and @abstractmethod
Instantiation	Not allowed	Not allowed	Not allowed
Concrete Methods	Allowed	Allowed	Allowed
Abstract Methods	Declared with = 0	Declared with abstract keyword	Declared with @abstractmethod
Inheritance	Derived classes must implement pure virtual functions	Subclasses must implement abstract methods	Subclasses must implement abstract methods
Modules/Headers	No special header needed	No special imports	Requires abc module

Interfaces

An interface is a programming construct that defines a contract or a set of methods that a class must implement. Interfaces are commonly used to achieve abstraction and multiple inheritance in java.

C++: Interfaces (pure abstract class)

- C++ does not have a built-in interface keyword, but interfaces can be implemented using abstract classes with only pure virtual functions.
- An interface in C++ is typically an abstract class with all methods declared as pure virtual (= 0).
- Classes implementing the interface must define all pure virtual functions.
- Multiple inheritance can be used to implement multiple interfaces.
- Interfaces cannot have any implementation (only declarations).

Subscribe Infeepedia youtube channel for computer science competitive exams
Download Infeepedia app and call or wapp on 8004391758

Syntax:

```
class Interface {
public:
   virtual void method1() = 0; // Pure virtual function
   virtual void method2() = 0;
   virtual ~Interface() {}   // Virtual destructor
};
```

Java

```
class Implementation implements
Interface {
   @Override
   public void method1() {
      System.out.println("Method1
implemented");
   }

   @Override
   public void method2() {
      System.out.println("Method2
implemented");
   }
}
public class Main {
   public static void main(String[] args) {
      Interface obj = new
Implementation();
      obj.method1(); // Output: Method1
implemented
      obj.method2(); // Output: Method2
implemented
   }
}
```

CPP

```
#include <iostream>
using namespace std;
class Interface {
public:
   virtual void method1() = 0;
   virtual void method2() = 0;
   virtual ~Interface() {}
};
class Implementation : public Interface
{
public:
   void method1() override {
      cout << "Method1 implemented"
<< endl;
   }
   void method2() override {
      cout << "Method2 implemented"
<< endl;
   }
};
int main() {
   Interface* obj = new
Implementation();
   obj->method1(); // Output:
Method1 implemented
   obj->method2(); // Output:
Method2 implemented
   delete obj;
   return 0;
}
```

Python

```
from abc import ABC, abstractmethod

class Interface(ABC):
   @abstractmethod
   def method1(self):
      pass

   @abstractmethod
   def method2(self):
      pass

class Implementation(Interface):
   def method1(self):
      print("Method1 implemented")

   def method2(self):
      print("Method2 implemented")

# Main Program
obj = Implementation()
obj.method1()  # Output: Method1
implemented
obj.method2()  # Output: Method2
implemented
```

Java: Interfaces

- Java provides a built-in interface keyword to define interfaces.
- An interface in Java is a reference type that contains abstract methods (by default) and constants.
- Starting from Java 8, interfaces can also have default methods and static methods.
- Methods in interfaces are public and abstract by default.
- Fields in interfaces are public, static, and final by default.
- A class can implement multiple interfaces.

Syntax:

```
interface Interface {
   void method1(); // Abstract method
   void method2();
}
```

Subscribe Infeepedia youtube channel for computer science competitive exams
Download Infeepedia app and call or wapp on 8004391758

```
interface Interface {
   void method1();
   void method2();
}
```

Python: Interfaces (abstract base class)

- Python does not have a built-in interface keyword, but interfaces can be implemented using the abc module (Abstract Base Classes).
- An interface in Python is defined as an abstract base class (ABC) with abstract methods.
- Abstract methods are declared using the @abstractmethod decorator.
- Classes implementing the interface must define all abstract methods.
- Python supports multiple inheritance, allowing classes to implement multiple interfaces.

Syntax:

```
from abc import ABC, abstractmethod
class Interface(ABC):
   @abstractmethod
   def method1(self):
      pass
   @abstractmethod
   def method2(self):
      pass
```

Comparison of Interfaces

Feature	C++	Java	Python
Definition	Abstract class with pure virtual methods	interface keyword	Abstract Base Class (ABC)
Keyword	No specific keyword	interface	@abstractmethod decorator
Multiple Interfaces	Supported via multiple inheritance	Supported	Supported
Default Methods	Not allowed	Allowed (from Java 8)	Not directly supported
Static Methods	Not allowed	Allowed (from Java 8)	Allowed in Python classes
Fields	No restrictions	public static final by default	No special restrictions

Abstraction

- Abstraction is a fundamental concept in object-oriented programming (OOP) that hides implementation details and exposes only essential features of an object.
- It focuses on what an object does rather than how it does it.

C++: Abstraction

- Achieved using abstract classes and pure virtual functions.
- Hides the implementation details of methods and enforces derived classes to provide their own implementations.
- A class with at least one pure virtual function (= 0) is abstract.
- Abstract classes cannot be instantiated.
- Abstraction can also be achieved using encapsulation by making members private or protected.

Syntax:

```
class AbstractClass {
public:
   virtual void abstractMethod() = 0; // Pure virtual function
   void concreteMethod() {
      cout << "Concrete method in abstract class" << endl;
   }
};
```

Java: Abstraction

- Achieved using abstract classes and interfaces.
- Abstract classes can have both abstract methods (without a body) and concrete methods (with a body).
- Interfaces define a contract that implementing classes must follow.
- Abstract classes are declared using the abstract keyword.
- Interfaces are declared using the interface keyword.
- Abstract classes and interfaces cannot be instantiated.

Syntax:

```
abstract class AbstractClass {
    abstract void abstractMethod(); // Abstract method
    void concreteMethod() {
        System.out.println("Concrete method in abstract class");
    }
}
```

Python: Abstraction

- Achieved using the abc module (Abstract Base Class).
- Abstract methods are declared using the @abstractmethod decorator.
- Abstract classes cannot be instantiated.
- Subclasses must implement all abstract methods.
- Python does not have an abstract keyword; abstraction is implemented through abc.

Syntax:

```
from abc import ABC, abstractmethod
class AbstractClass(ABC):
    @abstractmethod
    def abstract_method(self):
        pass
    def concrete_method(self):
        print("Concrete method in abstract class")
```

Java	CPP	Python

abstract class Shape {
  abstract void draw(); // Abstract method
}
 	
class Circle extends Shape {
  @Override
  void draw() {
  System.out.println("Drawing Circle");
  }
}
 	
public class Main {
  public static void main(String[] args) {
  Shape shape = new Circle();
  shape.draw(); // Output: Drawing Circle
  }
}
	#include <iostream>
	using namespace std;
	class Shape {
	public:
	  virtual void draw() = 0; // Pure virtual function
	};
	class Circle : public Shape {
	public:
	  void draw() override {
	  cout << "Drawing Circle" << endl;
	  }
	};
	int main() {
	  Shape* shape = new Circle();
	  shape->draw(); // Output: Drawing Circle
	  delete shape;
	  return 0;
	}
		from abc import ABC, abstractmethod
		
		class Shape(ABC):
		  @abstractmethod
		  def draw(self):
		  pass
		
		class Circle(Shape):
		  def draw(self):
		  print("Drawing Circle")
		
		# Main Program
		shape = Circle()
		shape.draw() # Output: Drawing Circle

Comparison of Abstraction

Feature	C++	Java	Python
Definition	Achieved using abstract classes	Achieved using abstract classes and interfaces	Achieved using abc module
Instantiation	Abstract classes cannot be instantiated	Abstract classes and interfaces cannot be instantiated	Abstract classes cannot be instantiated
Abstract Methods	Declared using = 0	Declared with abstract keyword	Declared with @abstractmethod
Concrete Methods	Allowed in abstract classes	Allowed in abstract classes	Allowed in abstract classes
Interfaces	No dedicated interface keyword; use abstract classes	Use interface keyword	No dedicated interface keyword; use abc
Multiple Inheritance	Supported	Supported via interfaces	Supported

C++: this Keyword

- this is a pointer that refers to the current instance of the class.
- It is implicitly passed to all non-static member functions of a class.
- this is not available in static member functions because they are not tied to any specific object.
- Accessing members of the current object.
- Resolving naming conflicts between instance variables and method parameters.
- Returning the current object (e.g., for method chaining).

Java: this Keyword

- this is a reference to the current instance of the class.
- It is implicitly available in all non-static methods and constructors.
- this is not accessible in static methods.
- Referring to instance variables when they are shadowed by method or constructor parameters.
- Invoking other constructors in the same class using this() (constructor chaining).
- Passing the current object as an argument to a method or constructor.

Python: self Keyword

- Python uses self instead of this.
- It refers to the current instance of the class and is explicitly passed as the first parameter to instance methods.
- In Python, self is not a keyword but a convention.
- Accessing instance variables and methods of the current object.
- Resolving naming conflicts between instance variables and method parameters.

Java

```
class Person {
   private String name;
   // Constructor
   Person(String name) {
      this.name = name; // Resolves
naming conflict
   }
   void display() {
      System.out.println("Name: " +
this.name); // Accessing the current
object's member
   }
   Person getThis() {
```

CPP

```
#include <iostream>
using namespace std;

class Person {
private:
   string name;
public:
   Person(string name) {
      this->name = name; // Resolves
naming conflict
   }
   void display() {
cout << "Name: " << this->name << endl;
```

Python

```
class Person:
   def __init__(self, name):
      self.name = name  # Resolves
naming conflict

   def display(self):
      print(f"Name: {self.name}")  #
Accessing the current object's member

   def get_self(self):
      return self  # Returning the current
object
```

Subscribe Infeepedia youtube channel for computer science competitive exams
Download Infeepedia app and call or wapp on 8004391758

```
        return this; // Returning the current object
    }
}
public class Main {
    public static void main(String[] args) {
        Person p = new Person("Alice");
        p.display(); // Output: Name: Alice
        System.out.println("Address of object: " + p.getThis());
    }
}
```

```
// Accessing the current object's member
    }

    Person* getThis() {
        return this; // Returning the current object
    }
};

int main() {
    Person p("Alice");
    p.display(); // Output: Name: Alice
    cout << "Address of object: " << p.getThis() << endl;
    return 0;
}
```

```
# Main Program
p = Person("Alice")
p.display()  # Output: Name: Alice
print(f"Address of object: {p.get_self()}")
```

Comparison of this/self in C++, Java, and Python

Feature	C++ this	Java this	Python self
Definition	Pointer to the current object	Reference to the current object	Reference to the current object
Implicit/Explicit	Implicit in all non-static methods	Implicit in all non-static methods	Explicit in method definitions
Static Method Access	Not available	Not available	Not applicable (explicit in Python)
Constructor Usage	Not used for constructor chaining	Used for constructor chaining (this())	Not used for constructor chaining
Naming	Keyword	Keyword	Convention (not a keyword)
Key Purpose	Access members and resolve conflicts	Access members, resolve conflicts, constructor chaining	Access members, resolve conflicts

super Keyword

The super keyword (in java) is used to refer to the parent class of the current class. It allows access to the parent class's methods, constructors, or attributes that are overridden or shadowed in the child class.

C++: Base Class Access

- C++ does not have a super keyword.
- Instead, the name of the parent (base) class is used explicitly to access overridden methods or constructors.
- Call the base class's constructor from the derived class.
- Access base class members or methods that are overridden in the derived class.

Java: super Keyword

- super is a keyword in Java used to refer to the immediate parent class.
- It is used in constructors and methods to access the parent class's methods, attributes, or constructors.
- Call the parent class's constructor using super().
- Access parent class methods or attributes that are overridden or shadowed in the child class.

Subscribe Infeepedia youtube channel for computer science competitive exams
Download Infeepedia app and call or wapp on 8004391758

Python: super Keyword

- super() is a built-in function in Python used to refer to the parent class.
- It is commonly used in class inheritance to access parent class methods, attributes, or constructors.
- Call the parent class's __init__ method in the child class constructor.
- Access parent class methods that are overridden in the child class.

Comparison of super Keyword

Feature	C++ (Base Class Access)	Java super	Python super()
Keyword	No super keyword, use base class name	super keyword	super() function
Constructor Call	Explicitly call base class constructor	Use super() in the constructor	Use super().__init__()
Method Access	Use base class name explicitly	Use super.methodName()	Use super().methodName()
Attribute Access	Use base class name explicitly	Use super.attributeName	Use super().attributeName
Dynamic Dispatch	No	Yes	Yes

Final keyword

The final keyword (or its equivalent in Python) is used to prevent further modifications in terms of inheritance or overriding. Its usage and behavior vary across C++, Java, and Python.

C++: final

- Introduced in C++11, the final keyword prevents a class from being inherited or a virtual method from being overridden in derived classes.
- Mark a class as final to prevent inheritance.
- Mark a virtual function as final to prevent overriding in derived classes.

Java: final

- The final keyword in Java can be applied to variables, methods, and classes.
- It prevents variables from being reassigned, methods from being overridden, and classes from being inherited.
- Mark a class as final to prevent inheritance.
- Mark a method as final to prevent overriding.
- Mark a variable as final to make it a constant (its value cannot be changed after initialization).

Python: final (via @final Decorator)

- Python does not have a final keyword but provides the @final decorator in the typing module (introduced in Python 3.8).
- The @final decorator is used to mark methods or classes as final, signaling that they should not be overridden or inherited.
- Mark a method as @final to prevent overriding.
- Mark a class as @final to prevent inheritance.

Comparison of final in C++, Java, and Python

Feature	C++ final	Java final	Python @final
Prevent Class Inheritance	final before class name	final before class name	@final decorator on class
Prevent Method Overriding	final after method name	final before method name	@final decorator on method
Prevent Variable Reassignment	Not applicable	final before variable	Not natively supported (use constants)
Availability	Since C++11	Always	Since Python 3.8

Inline Functions

C++: Inline Functions

- In C++, an inline function is a request to the compiler to replace the function call with the function's body to reduce function call overhead.
- It is defined using the inline keyword.
- Used for small functions to improve performance.
- The compiler may ignore the inline request if the function is too complex.
- Defined either inside the class or using the inline keyword.

- Inline functions reduce function call overhead and improve program performance.
- They are suitable for small programs and are declared using the inline keyword.
- Inline functions save stack overhead during function calls.
- All functions defined inside a class are implicitly inline.
- Inline functions cannot be used with loops, static variables, switch, goto, or recursion.
- Virtual functions cannot be inline because they are resolved at runtime.
- Inline functions enable the compiler to perform context-specific optimizations.
- Excessive use of inline functions increases binary size and reduces instruction cache efficiency.
- Modifying inline function code can increase compile-time overhead.
- Inline functions are preferred over macros due to type safety and compiler management.
- Unlike macros, inline functions can access private members of a class.

Java: Inline Functions (Implicit with JVM)

- Java does not have an explicit inline keyword.
- The Just-In-Time (JIT) compiler in the JVM automatically performs method inlining for small, frequently called methods.
- The decision to inline is handled by the JVM during runtime.
- Methods defined inside an interface or as final or static are more likely to be inlined because they cannot be overridden.

Python: Inline Functions (Simulated with Lambda or JIT)

- Python does not have a native inline keyword.
- Functions can be inlined manually using lambda expressions for simple cases.
- Advanced optimizations like inlining can be achieved using libraries like Numba or PyPy.
- Lambdas provide a concise way to define small functions inline.
- For performance improvement, JIT compilers like Numba can be used.

```
class Main {
   static int add(int a, int b) { // JVM
may inline this method
      return a + b;    }
   public static void main(String[] args) {
      int result = add(5, 10); // JVM
decides if inlining is beneficial
      System.out.println("Result: " +
result);     }  }
```

```
#include <iostream>
using namespace std;
inline int add(int a, int b) {    return a +
b; }
int main() {
   int result = add(5, 10); // Function
body is inserted here
   cout << "Result: " << result << endl;
   return 0;
}
```

```
add = lambda a, b: a + b  # Inline
function using lambda
result = add(5, 10)
print("Result:", result)
def add(a, b):
   return a + b
result = add(5, 10)  # Function call (no
true inlining in Python)
print("Result:", result)
Using Numba for JIT Optimization:
from numba import jit
@jit(nopython=True)
def add(a, b):
   return a + b
result = add(5, 10)  # JIT compiler
optimizes the function
print("Result:", result)
```

Comparison of Inline Functions

Feature	C++	Java	Python
Keyword	inline	No explicit keyword	No explicit keyword
Inlining Control	Programmer-defined (inline)	Handled by JVM (JIT)	Simulated with lambdas or JIT
Compiler Behavior	Compiler decides to inline	JVM decides to inline	No native compiler support
Use for Small Functions	Yes	Yes	Yes (via lambdas or JIT)
Support for Recursion	Not suitable	JVM avoids inlining recursion	No native support

Static Keyword Variable, Function, and Class in C++, Java, and Python

The concept of static is used to define variables, functions, or classes that are associated with a class rather than an instance. **Its behavior varies across C++, Java, and Python.**

In C++: Static variable and functions

Static variable

- Static variables are declared using the static keyword and are initialized to zero by default.
- They can be declared inside functions or classes.
- When declared in a function, static variables retain their value throughout the lifetime of the program.
- In a class, only one copy of a static variable is created, and it is shared by all objects of the class.
- Static variables are visible only within their defined scope.
- They are initialized before any class object is created, even before the main() function starts.
- Static variables cannot be initialized using constructors.
- A static variable in a class must be explicitly initialized outside the class using the class name and the scope resolution operator (::).
- Static objects also exist for the entire lifetime of the program.

Static Member Functions:

- Static member functions can only access static variables or other static functions within the class.
- They cannot access non-static variables or member functions.
- Static functions are only accessible within the class in which they are defined.

Uses of Static:

- To define variables with static storage duration and internal linkage.
- To declare block-scope variables that are initialized only once.
- To define class members that are shared across all instances of the class.

```
#include <iostream>
using namespace std;
class Test {
public:
   static int count; // Static variable declaration
   Test() { count++; } // Increment static variable
};
int Test::count = 0; // Static variable definition
int main() {
   Test t1, t2, t3;
   cout << "Count: " << Test::count << endl; // Access
using class name
   return 0;
}
```

```
#include <iostream>
using namespace std;
class Test {
public:
   static void display() { // Static function
  cout << "Static function called" << endl;
   }
};
int main() {
   Test::display(); // Call using class name
   return 0;
}
```

In Java: Static Keyword

Static Variable

- It is declared using the static keyword.
- Shared among all objects of the class (only one copy exists for the class).
- Initialized only once when the class is loaded into memory.
- Commonly used for constants or shared data.

Static Method

- A static method belongs to the class and cannot access non-static members directly.
- It is invoked using the class name.
- It can be called without creating an object of the class.
- Commonly used for utility or helper methods.

Static Class

- Java does not allow top-level classes to be static, but nested classes can be declared static.
- Useful for grouping utility methods.

Static Blocks

- Used to initialize static variables.
- Executes only once when the class is loaded into memory.
- Runs before the main() method or any static methods.

```
class Example
 { static int value;
static
{ value = 100; // Static block
System.out.println("Static block executed"); } }
```

Static variable	Static function	Static Class

class Test {
  static int count = 0; // Static variable
  Test() {
  count++;
  }
}
public class Main {
  public static void main(String[] args) {
  new Test();
  new Test();
  System.out.println("Count: " + Test.count); // Access using class name
  }
}
	class Test {
	  static void display() { // Static method
	  System.out.println("Static method called");
	  }
	}
	public class Main {
	  public static void main(String[] args) {
	  Test.display(); // Call using class name
	  }
	}
		class Outer {
		  static class Inner { // Static nested class
		  void display() {
		  System.out.println("Static nested class method");
		  }
		  }
		}
		public class Main {
		  public static void main(String[] args) {
		  Outer.Inner obj = new Outer.Inner(); // Create instance of static nested class
		  obj.display();
		  }
		}

In Python: Static Keyword

Python does not have a static keyword like Java or C++, but it provides similar functionality using class variables, staticmethods, and class methods.

Static Variables (Class Variables)

- Variables defined at the class level, shared across all instances of the class.
- Changes to a static variable affect all instances of the class.

Static Method

- Defined using the @staticmethod decorator.
- Do not require access to instance or class-specific data.
- Can be called using the class name or an instance.

Static Class

- Python does not have a static class keyword, but a class can contain only static methods and variables, making it effectively static.
- Used for utility functions or constants.

```
class Test:
    count = 0  # Static variable
    def __init__(self):
        Test.count += 1  # Access using class name

t1 = Test()
t2 = Test()
print("Count:", Test.count)  # Access using class name
```

```
class Test:
    @staticmethod
    def display():  # Static method
        print("Static method called")

Test.display()  # Call using class name
```

```
class StaticClass:
    @staticmethod
    def utility_function():
        print("Utility function in static class")

StaticClass.utility_function()
```

Feature	C++	Java	Python
Static Variable	Shared among instances, defined with static.	Shared among instances, defined with static.	Shared among instances, defined at class level.
Static Function	Defined with static, cannot access non-static members.	Defined with static, cannot access non-static members.	Defined with @staticmethod, cannot access instance variables.
Static Class	No keyword, can create classes with only static members.	Top-level classes cannot be static, but nested classes can.	No keyword, can simulate with static methods and variables.

Constants in C++

Constant Variable

A variable declared with the const keyword. Its value must be initialized at the time of declaration and cannot be modified after initialization.

The const keyword can be used with pointers to make the value or the pointer itself constant.

Syntax:

const <data_type> <variable_name> = <value>;

```
const int x = 10; // Constant variable x = 20; // Error: Cannot modify a const variable
```

```
const int* ptr = &x; // Data is constant, pointer can change
*ptr = 20; // Error: Cannot modify the data
 ptr = &y; // Valid: Pointer can point to another variable
```

Constant Pointers

The const keyword can be used with pointers to make the value or the pointer itself constant.

Pointer to Constant Data	Constant Pointer to Data	Constant Pointer to Constant Data

const int* ptr = &x; // Data is constant, pointer can change
*ptr = 20; // Error: Cannot modify the data
ptr = &y; // Valid: Pointer can point to another variable
	int* const ptr = &x; // Pointer is constant, data can change
	ptr = &y; // Error: Cannot change the pointer
	*ptr = 20; // Valid: Data can be modified
		const int* const ptr = &x; // Both pointer and data are constant
		*ptr = 20; // Error: Cannot modify the data
		ptr = &y; // Error: Cannot change the pointer

Constant Function

- Declared with const after the function name.
- Ensures the function does not modify any member variables of the class.
- **Syntax:**
  <return_type> <function_name>() const;

Constant Function Parameters: Prevents modification of arguments passed to a function.

Constant Return Values: Ensures the returned value cannot be modified by the caller.

Constant Objects: Objects declared as const can only call const member functions.

Constant Function	Constant Function Parameters	Constant Return Values	Constant Objects

class Example
{ int x;
public: void display() const
{
x = 20; // Error: Cannot modify member variables
std::cout << x << std::endl;
} };
	void display(const int value) {
	// value = 20; // Error: Cannot modify a const parameter
	std::cout << value << std::endl;
	}
		const int getValue() {
		return 10;
		}
		int x = getValue();
		x = 20; // Valid: x is not const
			const Example obj;
			// obj.modify(); // Error: Cannot call non-const function
			obj.display(); // Valid: Can call const function

Final (Constants) in Java

- Java does not have a direct const keyword; the final keyword is used to create constants in Java.
- A final variable must be initialized once and cannot be changed.
- static final is used for global constants shared across all instances.
- final methods cannot be overridden, and final classes cannot be extended.

Constant Variables	Static Final Constants	Final Parameters	Final Methods

class Example {
final int CONSTANT_VALUE = 100; // Constant variable
void display()
// CONSTANT_VALUE = 200; // Error: Cannot reassign a final variable
System.out.println(CONSTANT_VALUE); } }
	class Example {
	static final double PI = 3.14159; // Static constant
	void display() {
	System.out.println("Value of PI: " + PI);
	}
	}
		class Example {
		void display(final int value) {
		// value = 20; // Error: Cannot modify final parameter
		System.out.println(value);
		}
		}
			class Parent {
			final void display() {
			System.out.println("This is a final method");
			} }
			class Child extends Parent {
			// void display() { }
			// Error: Cannot override a final method }

Final Classes: Prevents a class from being inherited.

Example:

```
final class Example {
   void display() {
      System.out.println("This is a final class");
   }
}
// class SubExample extends Example { } // Error: Cannot inherit from a final class
```

Constants in Python

- Python does not have a built-in const or final keyword.
- By convention, constants are written in uppercase (e.g., PI, GRAVITY).
- Constants can be stored in separate modules for better organization.
- The Final annotation from typing (introduced in Python 3.8) provides an optional way to indicate constants but does not enforce immutability.
- Python constants rely on developer discipline rather than enforced immutability.

Constants variable	Constant Function	Constant Class

PI = 3.14 # Constant variable by convention
print("Value of PI:", PI)
# PI = 3.14159 # Allowed but discouraged 	
	def display(): # Function treated as constant
	print("This is a constant function.")
	display()
		from dataclasses import dataclass
		
		@dataclass(frozen=True)
		class Test:
		value: int
		
		t = Test(10)
		print("Value:", t.value)
		# t.value = 20 # Error: Cannot assign to field

Comparision of constant in C++, Java and Python

Feature	C++	Java	Python
Constant Variable	const keyword, value immutable.	final keyword, value immutable.	Naming convention (all-uppercase).
Constant Function	Declared with const, cannot modify state.	No direct equivalent; use final to prevent overriding.	Functions that avoid state modification.
Constant Class	No direct keyword, but members can be const.	Declared with final, cannot be subclassed.	Use @dataclass(frozen=True) for immutability.

Friend Function and Class in C++

Friend Class

- Declared using the friend keyword.
- Allows one class to access the private and protected members of another class.
- Can be declared in any access block (private, protected, or public).
- The friend keyword is used only in the declaration, not in the function definition or calls.

Example:

```
#include <iostream>
using namespace std;
class Car {
private:
   int x;
public:
```

```
    Car() { x = 45; }
    friend class Auto; // Declaring Auto as a friend class
};
class Auto {
public:
    void show(Car &obj) {
        cout << "Private Variable of Car class = " << obj.x << endl;
    }
};
int main() {
    Car c;
    Auto a;
    a.show(c);
    return 0;
}
// Output: Private Variable of Car class = 45
```

Friend Function

- A non-member function declared as a friend can access private and protected members of a class.
- The friend keyword is used only in the declaration, not in the definition or when calling the function.
- Friend functions are called like normal functions, not with an object and dot operator.
- It can be:
    1. A global function.
    2. A member function of another class.

Syntax:

```
friend return_type function_name(arguments); // Global function

friend return_type class_name::function_name(arguments); // Member function of another class
```

Friend Function and Class in Java

- Java doesn't have a friend keyword as in C++.
- Use package-private (default access) for friend-like functionality within the same package.
- Use protected for access in subclasses across packages.
- Java promotes encapsulation and controlled access using getter and setter methods, instead of directly exposing private members.
- By using these mechanisms, Java achieves similar functionality to friend functions and classes in a more controlled and secure manner.

Friend Function and Class in Python

Python does not have a friend keyword. However, similar functionality can be achieved through:
Access modifiers: Use naming conventions like _ (protected) or __ (private) for attributes.
Direct access: Python allows direct access to private attributes, as it does not enforce strict access restrictions.
In Python, friend-like functionality is achieved through dynamic access, naming conventions, and encapsulation.

Comparison of friend keyword in C++, Java and python

Feature	C++	Java	Python
Friend Function	friend keyword gives access to private data.	Use public getters or package-private access.	Access private attributes directly or via methods.
Friend Class	friend keyword allows class-level access.	Use package-private or protected access.	Access private attributes directly or via methods.